

Metadata Management for CephFS

Quick Objective: Improve the CephFS metadata bottlenecks (specially for small-file workloads) by combining proactive placement and small-file packing.

Project Updates: Over the past few days, I focused primarily on literature review and experimenting CephFS bottlenecks on differing workloads.

Insights From Literature Review:

I looked for papers discussing metadata placement and file-packing.

On the placement side, I looked at systems like Mantle (CephFS), Lunule+, and IndexFS/Giga+. Mantle was especially useful since it exposes a programmable interface for metadata placement in CephFS, showing that custom policies can outperform the default balancer. Haven't tried this out yet, but will be exploring this a bit more. Lunule+ and related work highlight that CephFS's subtree partitioning is still reactive and migration-heavy, motivating more predictive approaches. IndexFS/Giga+ show an alternative design (hash-based distribution) that achieves strong load balancing but sacrifices locality. I hope to balance load balancing while preserving locality. I would like to have some hash-based distribution as a baseline.

For small-file workloads, I read TableFS, DeltaFS, and HDFS HAR/AHAR. TableFS is pretty relevant since it stores metadata and small files in a KV store (LSM-tree), reducing metadata overhead significantly. DeltaFS extends this with log-structured ingest and compaction, which is very close to the packing layer I'm proposing. HAR/AHAR show a simpler "file grouping" approach but also make clear the tradeoff with reduced metadata pressure vs increased lookup overhead. I thought this suggested nice evaluation metrics around lookup amplification vs metadata savings.

I also need to look into HopsFS/HopsFS++, which showed good results colocating small-file data with metadata.

Bottleneck Experiments:

First for my experiment bed, I spun up a 4 node Cloudlab setup, with every other default parameter. I configured one node as a client-only benchmark driver and used the other three nodes as Ceph storage/metadata servers. Those three server nodes each hosted a monitor (mon), metadata server (mds), and object storage daemon (osd). Among the MDS daemons, two were active and one served as a standby, while the admin node was also used as the main control point for cluster management. Ceph also added a monitor to the client node by default, I just ran with it for now but it has minimal impact on performance since it doesn't use a lot of resources. For figures in my final paper, and for future runs, I will remove this monitor for complete consistency.

So far, my focus has been on running experiments that explore the motivation of my project and identify which workload patterns are most sensitive to CephFS metadata overhead. I ran synthetic workloads varying the following factors:

- Number of files while keeping total data size fixed (64MiB)
- Random vs Sequential Access
- Read vs Write Heavy Workloads

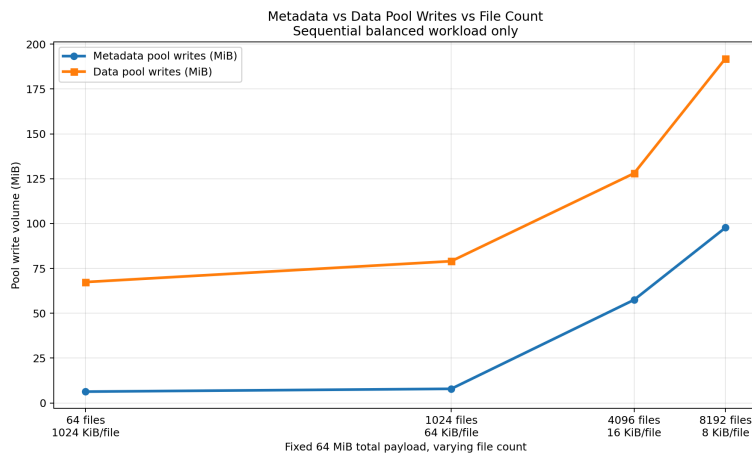
Each workload performed a full file lifecycle including create/write, stat, read, and delete operations, so the cluster experienced both metadata operations and real file I/O.

Further, the use of 64MiB for total data size was for quick prototyping and tests. I will be running a selected set of experiments for larger data sizes. Since we were just studying percentages for something like Ceph, I think an increase in size shouldn't have monumental differences for most of our metrics. Generally, in these workloads I tracked:

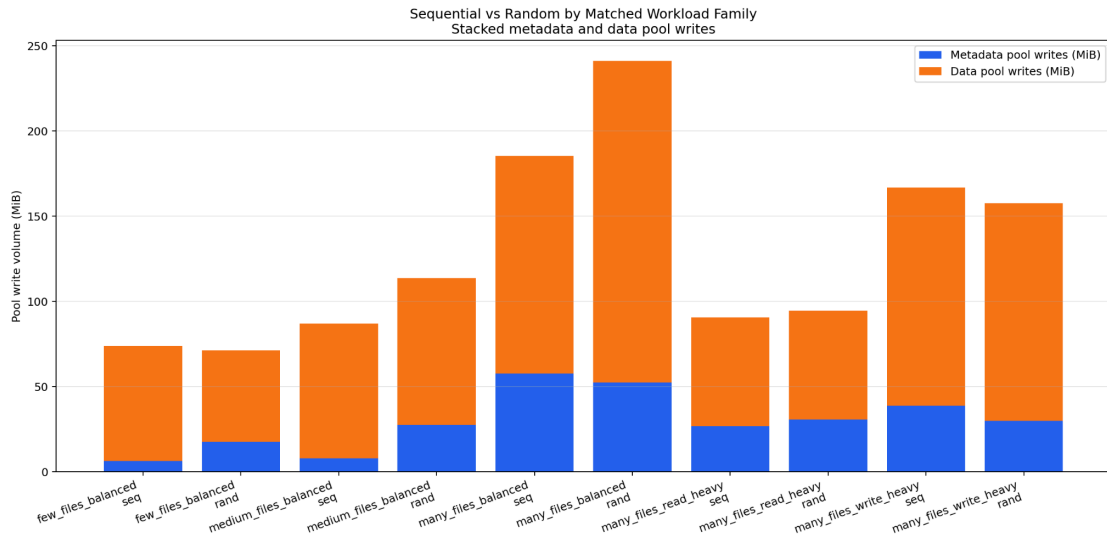
- Create, read, stat, and delete throughput
- Per-operation latency statistics, including p95/p99 where available
- Metadata-pool write volume versus data-pool write volume
- Distribution of requests across active MDS daemons
- The relative mix of metadata-style work versus payload I/O

The key idea behind the file-count sweep was to hold total payload constant while increasing the number of files, which shrinks file size and increases namespace pressure. Generally studying metadata pressure without adding the pressure associated with more data.

Some Relevant Results:



The jump with increasing number of files here in metadata pool writes is clearly visible and important here.



Here, essentially comparing the work being done for sequential and random workloads with different amounts of files and read/write workload (balanced represents something middle of the road b/w the two). Generally the overall workload seems similar but with more variation for a large number of files. Obviously actual throughput is different but this is more composition of the writing work. Here, the general metadata writes seem to be higher for a higher number of files which aligns with the overall intuition.